

Solution

YDSP 2024 云斗初赛模拟 J 组

tder

云斗出题组

2024 年 9 月 18 日

① Answer

② Part 1

③ Part 2

④ Part 3

① Answer

② Part 1

③ Part 2

④ Part 3

Part 1

- ACACB DADCD DCBAC

Part 2

- BAABCB BAABAD ABBDCD

Part 3

- BDAAD ADCDB

① Answer

② Part 1

③ Part 2

④ Part 3

1st ~ 3rd

- 【1】模拟法.
- 使用短除法, 有 $2024 = 2^3 \times 11 \times 23$.
- 【1】位、字节与字.
- 有 $1 \text{ B} = 8 \text{ bit}$, $1 \text{ KB} = 1024 \text{ B}$, $1 \text{ MB} = 1024 \text{ KB}$. 故 A 选项 $2024 \text{ B} \approx 1.98 \text{ KB}$, B 选项 $41 \text{ MB} = 41984 \text{ KB}$, C 选项 $0.04 \text{ MB} = 40.96 \text{ KB}$, D 选项 $114514 \text{ b} \approx 13.98 \text{ KB}$. 仅有 B 选项不满足条件.
- 【1】进制与进制转换: 八进制, 【2】位运算: 异或 ($\wedge$).
- 有 $(72)_8 = (0111010)_2$, 而 $(110)_{10} = (1101110)_2$. 故两数按位异或得 $(1010100)_2 = (84)_{10}$.

4th ~ 6th

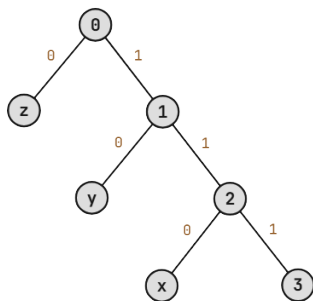
- 【3】 二叉树的定义与基本性质, 【3】 贪心法.
- 考虑尽可能挂在一条链上. 构造长为 1104 的链, 在深度为 $1 \sim 920$ 的点上各挂一个叶子. 此时叶子数为 $920 + 1 = 921$, 包括链底部的深度为 1104 的点.
- 【2】 乘法原理.
- 根据乘法原理, 总方案数为 $4 \times 4 = 16$ 种. 而不合法的方案包括两个组别都选了模拟, 或两个组别都选了 DP, 注意前者有两种方案, 即 CSP-J 选取第一道题或选取第三道题, 共 3 种方案. 故合法方案数为 $16 - 3 = 13$ 种.
- 【1】 表达式的概念.
- 化为中缀表达式为 $(4 - (3 - 4)) + ((6 \div 2) \times 1) = 8$.

7th ~ 8th

- **【4】指针，【4】向量（vector）。**
- 函数 `end()` 指向最后一个元素后的地址，可以认为是第 6 个元素，前移 3 个地址后指向第 3 个元素，故答案为 30.
- **【1】数组与数组下标，【2】string 类与相关函数。**
- 由于 string 类的下标从 0 开始，故函数等价于翻转 $s[l-1 \dots r-1]$. 设当前处理到第 i 个字符，它是子串左端点 $l-1$ 右边的第 $i-(l-1)$ 个字符，应当将它与子串右端点 $r-1$ 左边的第 $i-(l-1)$ 个字符交换，即第 $(r-1)-(i-(l-1)) = l+r-i-2$ 个字符.

9th

- 【4】 哈夫曼树的定义和构造、哈夫曼编码.
- 观察到 $2^i + 2^{i+1} < 2^{i+2}$, 构造出的哈夫曼树的一部分如下图所示, 以此类推. 故字符 e 的哈夫曼编码长度为 $26 - 5 + 1 = 22$.



10th

- **【3】** 算法模板库中的函数：sort，**【4】** 队列（queue）、链表（list）。
- A 选项 list 无法直接使用下标随机访问，因为其本质上是一个链表，可使用迭代器访问元素。B 选项 STL 中的队列 queue 需要用函数 front() 来取队首，函数 top() 为栈 stack 取栈顶元素的函数。C 选项 reserve() 函数用于保留内存，而非 reverse() 的翻转。D 选项正确。

11th ~ 12th

- 【4】 二叉树的遍历：前序、中序.
- 由前序遍历有 A ~ J 分别表示 3169208574，模拟得中序遍历为 9123058746.
- 【4】 排列.
- 男生和男生之间有区别，女生同理. 总方案数为 $7! = 5040$ 种. 不合法方案即所有男生都在女生前面，即前 3 个人都是男生，后 4 个人都是女生，共 $3! \times 4! = 144$ 种. 故合法方案共 $5040 - 144 = 4896$ 种.

13th ~ 14th

- 【3】 选择排序.
- 考虑第一次操作将所有牌都交给 Alice，确定其中最大牌和最小牌后，第二次操作将剩余牌交给 Alice，此时便可确定完整序列的次大牌和次小牌. 以此类推，仅需 3 次操作即可.
- 【1】 模拟法.
- 观察发现发生必然错误当且仅当在某个决策点上，当前较小数的路径更优，且较大数的路径与之必然无交. 因此，不妨构造：

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 2 & 0 & 0 \end{bmatrix}$$

15th

- 【3】 图的定义与相关概念.
- 情况 1 中, 这 20 个度数为 20 的点已有 $\frac{20 \times 20}{2} = 200$ 条边, 而总共仅有 200 条边, 则其它点的度数均为 0. 但即使是这 20 个点组成的完全图中, 每个点的度数也仅为 19, 因此不可能. 情况 3 中, 其它 10 个点的度数和为 $2 \times 200 - 90 = 310$. 除去与外部相连的至多 90 条边外, 其内部至少连了 220 条边. 由 10 个点组成的完全图的总边数为 $\frac{10 \times 9}{2} = 45$, 远小于 220, 因此不可能. 故仅有情况 2 可能.

1 Answer

2 Part 1

3 Part 2

16th

17th

18th

4 Part 3

1 Answer

2 Part 1

3 Part 2

16th

17th

18th

4 Part 3

16th

- **【1】** 字符型: char, **【2】** string 类与相关函数.
- 程序实现了弱化版的 P1079 Vigenère 密码.

16th (1st ~ 2nd)

- 【2】 ASCII 码.
- 观察 ASCII 码表发现, 小写字母 a 的 ASCII 码为 97, 小写字母 z 的 ASCII 码为 122, 大写字母 Z 的 ASCII 码为 90. 由于 $(90 - 97) + (122 - 97) = 17 > 0$, 当输入为 `Z\nz` 时, 程序不会出错.
- 【2】 ASCII 码.
- 在 ASCII 码表中, 大写字母在小写字母之前, 故若 `key` 中有大写字母, 用其 ASCII 码减去小写字母 a 的 ASCII 码可能导致结果为负数, 从而发生数组越界, 程序可能出错.

16th (3rd ~ 4th)

- 【2】 ASCII 码.
- 由程序第 18 行, `code` 数组中均为小写字母, 这是由于小写字母 a 至 z 的 ASCII 码为 97 至 122. 正常情况下, 调用 `encode()` 后 `s` 中均为 `code` 数组中的某个字符, 因此均为小写字母.
- 【1】 数组与数组下标.
- 考虑何时不能正常运行, 当且仅当调用 `s[i]` 或 `key[i]` 时出现越界. 而 $0 \leq i < |s|$, 故前者一定不会越界. 而当 $i \geq |key|$ 时后者出现越界, 导致程序异常退出. 于是当 $|s| \leq |key|$ 时程序正常运行.

16th (5th ~ 6th)

- 【1】 模拟法.
- 根据题意手玩即可. 注意程序中对 26 取模, 因此 z 后移 1 个字母为 a.
- 【1】 模拟法.
- 正向操作是将每个 s_i 后移 $(key_i - a)$ 个字母, 则反向操作即将每个 s_i 前移 $(key_i - a)$ 个字母.

1 Answer

2 Part 1

3 Part 2

16th

17th

18th

4 Part 3

17th

- 【1】循环结构, 【3】辗转相除法 (欧几里得算法), 【4】素数筛法: 埃氏筛法与线性筛法.
- 程序分为三个部分, 分别实现了暴力求逆元、辗转相减法求最大公约数、线性筛计算 n 以内素数个数.

17th (1st ~ 3rd)

- 【1】代数（初中部分）.
- 由于保证了 $\text{mod} \nmid n$, 任何正整数 n 都存在逆元, 因此不可能返回 -1 .
- 【3】辗转相除法（欧几里得算法）.
- 这是两种不同的辗转相减法的写法.
- 【4】素数筛法：埃氏筛法与线性筛法.
- 执行第 38 行意味着 i 之前已经被 prime_j 筛掉了, 由于 prime 中的素数是递增的, 因此 i 乘上其它素数一定会被 prime_j 的倍数筛掉, 故加上或去掉均不影响输出. 但是, 删除后程序的程序复杂度变为埃筛的 $\mathcal{O}(n \log n)$.

17th (4th ~ 6th)

- 【1】 模拟法.
- 根据题意手玩即可. 有 $27 \times 14 \equiv 378 \equiv 1 \pmod{29}$.
- 【3】 辗转相除法 (欧几里得算法) .
- 程序使用辗转相减法求最大公约数.
- 【1】 模拟法.
- 根据题意手玩即可. 有 $\gcd(78, 52) = 26$.

1 Answer

2 Part 1

3 Part 2

16th

17th

18th

4 Part 3

18th

- 【1】模拟法, 【1】代数 (初中部分), 【3】辗转相除法 (欧几里得算法) .
- 程序实现了将一个小数化为最简分数, 其中括号表示循环节. 程序中 $f_1 = 1$ 表示有限小数, $f_3 = 1$ 表示混循环小数, $f_2 = 1$ 表示其它无限循环小数.

18th (1st ~ 3rd)

- 【2】赋值语句.
- 由于第 13 行将 i 的初值设为 1, 若 $l = 0$ 第 16 行仍返回 1.
- 【1】模拟法.
- 根据题意手玩即可. 输出应为 707713249/349650, 注意中间有左斜杠.
- 【1】模拟法.
- 此时执行前 39 行语句后 $num = l = \{1, 0, 0\}$, $f_1 = 1$, $f_2 = f_3 = 0$. 继续执行 40 至 42 行得 $a = 1 \times 1 + 0 = 1$ 且 $b = 0$, 有 $\gcd(a, b) = 1$, 故输出 $1/1$. 程序不会发生运行错误.

18th (4th ~ 6th)

- 【1】 模拟法.
- 此时由于 $f_1 = f_2 = f_3 = 0$, a 和 b 的值均为 0, 这导致 $\gcd(a, b) = 0$, 从而在第 53 行调用 $a / \gcd(a, b)$ 时发生除以 0 运行错误, 因此程序无输出.
- 【1】 模拟法.
- 此时 $f_1 = f_2 = 0$, $f_3 = 1$, 故会执行第 47 至 50 行.
- 【1】 模拟法.
- 根据题意手玩即可.

① Answer

② Part 1

③ Part 2

④ Part 3

19th

20th

① Answer

② Part 1

③ Part 2

④ Part 3

19th

20th

19th

- 【4】 动态规划的基本思路，【4】 简单一维动态规划，【5】 简单背包类型动态规划.
- 程序的时间复杂度为 $\mathcal{O}(V \cdot n)$.

19th (1st ~ 3rd)

- 【4】简单一维动态规划.
- 程序在计算 $f(x)$ 部分采取暴力转移 $f(i + \lfloor \frac{i}{j} \rfloor) \leftarrow f(i) + 1$.
- 【0】阅读理解题意.
- 与下一问一起考虑. 由 $f(x)$ 的定义, 若可操作次数 $k \geq \sum f(b_i)$, 即所有 a_i 均能变为 b_i , 则答案为所有 c_i 之和.
- 【0】阅读理解题意.
- 见上.

19th (4th ~ 5th)

- 【5】 简单背包类型动态规划.
- 与下一问一起考虑. 这是一个朴素的 01 背包. 有 n 个物品, 其体积和价值分别为 $f(b_i)$ 和 c_i . 通过倒序枚举第二维的方式, 压掉一维空间. 有转移 $g(j) \leftarrow g(j - w_i) + v_i$.
- 【5】 简单背包类型动态规划.
- 见上.

① Answer

② Part 1

③ Part 2

④ Part 3

19th

20th

20th

- **【4】二分法，【5】深度优先搜索.**
- 程序通过二分答案，使用 DFS 进行检验答案是否可行.

20th (1st ~ 3rd)

- 【5】 深度优先搜索.
- 若当前点已被访问过, 则不可行.
- 【5】 深度优先搜索.
- 若当前边 $u \rightarrow v$ 的边权 w 不超过二分的最大边权 $\max$, 则可以向下 DFS, 最大边权不变, 当前点变为 v . 若向下 DFS 的路径 $v \rightsquigarrow t$ 可行, 则 $u \rightarrow v \rightsquigarrow t$ 也可行.
- 【5】 深度优先搜索.
- 若不存在 $u \rightsquigarrow t$ 的路径, 则不可行. 此处不应回溯, 这是由于 check 只需验证是否合法, 而不需要构造最优解. 回溯会导致复杂度增加.

20th (4th ~ 5th)

- 【4】 二分法.
- 答案上界应为边权的最大值, 即 10^6 .
- 【4】 二分法.
- 常规写法应为 $mid \leftarrow mid + 1$. 但发现 B 选项亦可通过, 这是因为任何正整数都能写成 $a \times 2^k$ 的形式, 不断二分总可使区间长度为奇数.

Thank you!